BVM ENGINEERING COLLEGE
(An Autonomous Institution)
Vallabh Vidyanagar – 388120
Electronics and Communication Department

EcoLoop

E-Waste Collection Management System

Developed in C++ using Object Oriented Programming

Subject: 204ES – Object Oriented Programming
Academic Year: 2026–27, Semester III

Submitted by Pranshu Parikh
ID No. 25EC419
Batch A
Faculty Prof. Mayur Sevak

CERTIFICATE

This is to certify that **Pranshu Parikh**, ID No. **25EC419**, Batch A, of the Electronics and Communication Department, BVM Engineering College, Vallabh Vidyanagar, has satisfactorily completed the individual project titled **"EcoLoop: E-Waste Collection Management System"** in the subject **204ES: Object Oriented Programming** for the academic year 2026–27.

Date: _____ Signature of Faculty: _____

Abstract

Electronic waste is among the fastest-growing waste streams in the world, yet most people do not know where or how to dispose of old phones, chargers, batteries and appliances safely. **EcoLoop** is a menu-driven console application written in C++ that connects users with e-waste collection. A user can register, request a pickup for a set of e-waste items, track the pickup status, and earn reward points based on the type and weight of the e-waste handed over; an administrator can review and complete pickups. The project is designed to apply the main concepts of Object Oriented Programming: classes and objects, encapsulation, inheritance, abstract classes and polymorphism, operator and function overloading, static members, friend functions, templates, the Standard Template Library, exception handling and file handling.

Contents

1 Introduction **4**

2 Problem Statement **4**

3 Objectives **4**

4 Scope and Features **5**

4.1 Functional features....5

4.2 Limitations of scope....5

5 OOP Concepts Applied **5**

6 System Design **6**

6.1 Class diagram....6

6.2 Description of classes....7

6.3 Reward points rule....8

6.4 Program flow....8

6.5 Files and modules....8

7 Key Code Fragments **9**

7.1 Abstract base class and inheritance....9

7.2 Operator overloading for reward points....9

7.3 Static member and exception handling....9

7.4 Function template....10

8 Testing Plan **10**

9 Output Screenshots **11**

10 Advantages and Limitations **11**

10.1 Advantages....11

10.2 Limitations....11

11 Future Scope **11**

1 Introduction

Electrical and electronic equipment has become part of everyday life, and the devices are replaced at an ever faster rate. When they reach the end of their life they become *e-waste*. According to the Global E-waste Monitor 2024, the world generated about 62 million tonnes of e-waste in 2022, of which only 22.3% was documented as properly collected and recycled, and the amount is projected to reach about 82 million tonnes by 2030 [1]. E-waste contains valuable materials such as copper, gold and rare metals, but also hazardous substances such as lead, mercury and cadmium, so improper disposal harms both the environment and health.

A major reason for low recycling rates is the lack of awareness and convenience: citizens often keep old devices at home or hand them to informal scrap dealers because they do not know of an easy, trustworthy channel. EcoLoop addresses this by giving users a single place to register, schedule a pickup and be rewarded for responsible disposal.

2 Problem Statement

People do not know where or how to dispose of their e-waste, and there is no simple way to schedule its collection, track it or be motivated to do so. The aim of this project is to design and implement an object oriented C++ system that lets users register, schedule e-waste pickups, earn reward points and view their recycling history, while allowing an administrator to manage the pickups.

3 Objectives

- To model the real-world entities of e-waste collection (users, administrators, items, pickups and rewards) as C++ classes.
- To allow users to register and to schedule e-waste pickups.
- To reward users with points calculated from the category and weight of the e-waste.
- To maintain a recycling history for every user.
- To store data permanently in files so that it is available in later runs of the program.
- To apply advanced OOP concepts in a clean, reusable and extensible design.

4 Scope and Features

4.1 Functional features

1. **User registration and login** – name, phone number, address and a unique user ID.
2. **Schedule pickup** – the user selects item categories, enters weights and chooses a pickup date.
3. **Reward points** – points are credited when a pickup is completed.
4. **Recycling history** – a list of all pickups with their date, items, weight and status.
5. **Admin panel** – view pending pickups, mark them as completed or cancelled, and view overall statistics.
6. **Persistent storage** – users and pickups are saved to and loaded from files.

4.2 Limitations of scope

The system is a console application for a single machine; it does not include a graphical interface, online payment or real-time maps.

5 OOP Concepts Applied

Table 1 shows where each concept is used in the design.

6 System Design

6.1 Class diagram

Figure 1 shows the main classes and their relationships.

Concept	Use in EcoLoop
Classes and objects	User, Adnin, EvasteIten, Pickup, RevardAccount
Encapsulation	Data members are private; accessed through getters and validated setters
Inheritance	User and Adnin derive from the base class Person
Abstract class and polymorphism	Person has a pure virtual function displayProfile() , overridden by each derived class and called through base-class pointers
Constructors and destructors	Initialisation of objects; the destructor of the system class saves data to file
Function overloading	schedulePickup() with and without a preferred date
Operator overloading	+= on RevardAccount to add points; śś to print items and pickups
Static members	A static counter in EvasteIten and a static ID generator in Pickup
Friend function	A friend function that prints the full report of a User
Templates	A generic function template for searching an ID in a vector of any class
STL	vector for lists, nap for the category-to-rate table
Exception handling	A custom InvalidInputException for invalid weight, date or ID
File handling	ifstrean/ofstrean to save and load users and pickups

Table 1: OOP concepts and their use in the project

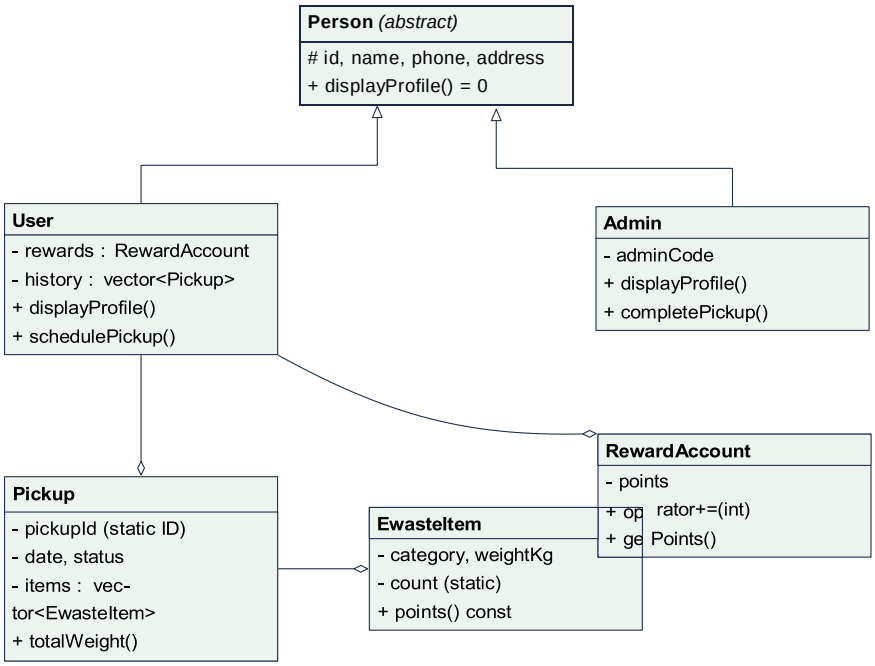


Figure 1: Class diagram of EcoLoop (open triangle: inheritance; open diamond: aggregation)

6.2 Description of classes

Person **(abstract base class)**

Holds the common data of every person in the system: ID, name, phone number and address. Declares the pure virtual function displayProfile() .

User

Derived from Person. Stores the user's RevardAccount and list of pickups, and provides functions to schedule a pickup and to view history.

Adnin

Derived from Person. Can list pending pickups and mark a pickup as completed or cancelled.

EvasteIten

Represents one item: category (for example mobile phone, battery, laptop, large appliance) and weight in kilograms. Computes the reward points of the item from the category rate.

Pickup

Represents one collection request: unique pickup ID, date, status (*Pending*, *Completed*, *Cancelled*) and the list of items.

RevardAccount

Keeps the user's points. The operator += adds points after a pickup is completed.

EcoLoopSysten

The controller class: shows the menus, owns the lists of users and pickups, and

loads and saves the files.

6.3 Reward points rule

Points for an item are points = weight (kg) × rate of its category. The rates in Table 2 are proposed values kept in a `nap` so that they can be changed easily.

Category	Points per kg
Batteries	20
Small gadgets (phones, chargers, earphones)	15
IT equipment (laptops, keyboards, printers)	12
Large appliances	8
Other electronic items	5

Table 2: Proposed reward rates

6.4 Program flow

1. Start the program and load users and pickups from the data files.
2. Show the main menu: *Register*, *User Login*, *Admin Login*, *Exit*.
3. **User menu:** schedule a pickup, view history, view reward points, view profile, logout.
4. **Admin menu:** list pending pickups, complete a pickup (adds points to the user), cancel a pickup, view statistics, logout.
5. On exit, save all data to the files.

6.5 Files and modules

File	Purpose
`nain. cpp`	Entry point; creates `EcoLoopSysten` and runs the menu loop
`Person. h, User. h, Adnin. h`	Person hierarchy
`EvasteIten. h, Pickup. h`	Item and pickup classes
`RevardAccount. h`	Reward points handling
`EcoLoopSysten. h`	Controller, menus and file handling
`users. txt, pickups. txt`	Persistent data files

Table 3: Proposed project structure

7 Key Code Fragments

The following fragments show how the central OOP ideas are expressed in code.

7.1 Abstract base class and inheritance

```
class Person (
protected:
    int id;
    string name, phone, address;
public:
    Person(int i, string n, string p, string a)
        : id(i), name(n), phone(p), address(a) ()
    virtual void displayProfile() const = 0; // pure vìríuaí
    virtual *Person() ()
);

class User : public Person (
    RevardAccount revards;
    vectoršPickup3 history;
public:
    User(int i, string n, string p, string a) : Person(i, n, p, a) ()
    void displayProfile() const override (
        cout šš "User #" šš id šš ": " šš name šš " (" šš phone šš ")\n";
    )
);
```

7.2 Operator overloading for reward points

```
class RevardAccount (
    int points = 0;
public:
    RevardAccount& operator+=(int p) ( points += p; return *this; )
    int getPoints() const ( return points; )
);
```

7.3 Static member and exception handling

```
class EvasteItem (
    string category;
    double veightKg;
    static int count; // number of ìíems creaíed
public:
    EvasteItem(string c, double v) : category(c), veightKg(v) (
        if (v š= 0) throv invalid_argument("Weight must be positive");
        ++count;
    )
    static int getCount() ( return count; )
);
```

```
int EvasteItem::count = 0;
```

7.4 Function template

```
template šclass T3
T* findById(vectoršT3& list, int id) (
    for (auto& obj : list)
        if (obj.getId() == id) return &obj;
    return nullptr;
)
```

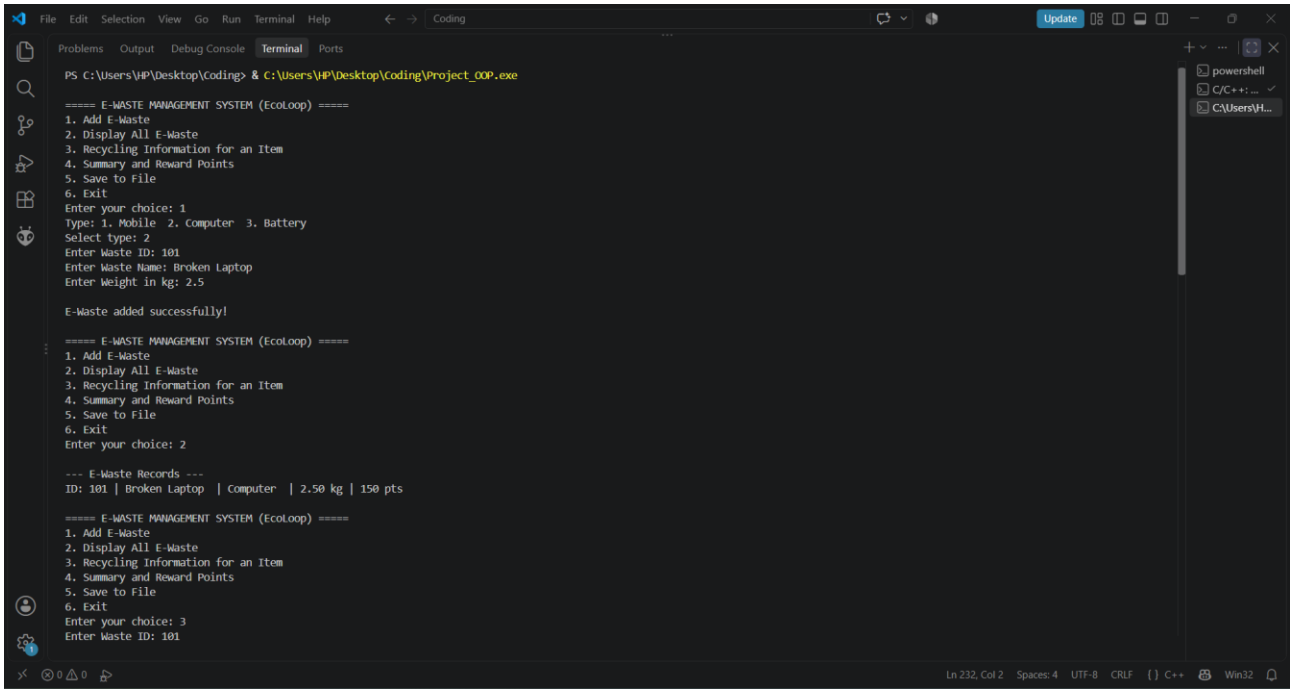
8 Testing Plan

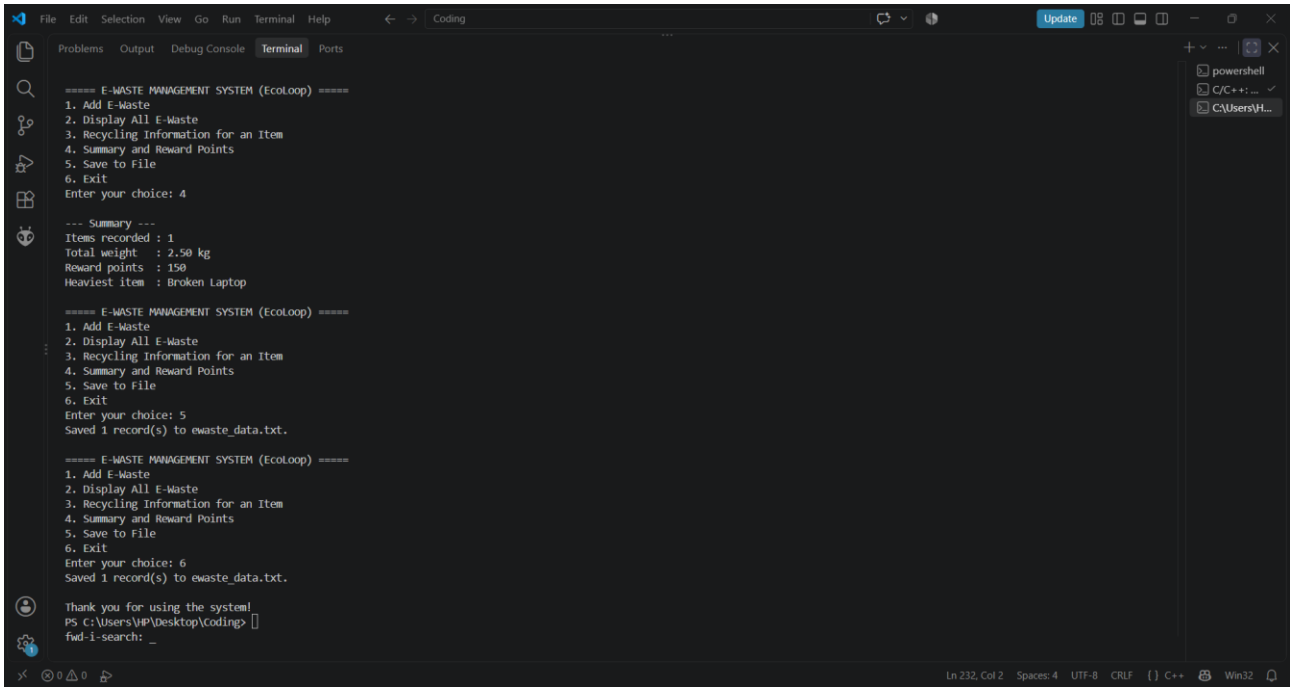
Table 4 lists the main test cases and the expected behaviour of the system.

No.	Test case	Expected result
1	Register a new user	User gets a unique ID and is saved to the file
2	Login with an unknown ID	Error message; user stays on the main menu
3	Schedule a pickup with 2 kg of batteries	Pickup is created with status Pending
4	Enter a negative or zero weight	Exception is caught; item is rejected
5	Admin completes a pickup of 2 kg batteries	40 points are credited using the rate 20 points/kg
6	Cancel a pending pickup	Status becomes Cancelled; no points are credited
7	View history	All pickups of that user are listed with date, items and status
8	Exit and restart the program	Users, pickups and points are reloaded from the files

Table 4: Test cases

9 Output Screenshots





10 Advantages and Limitations

10.1 Advantages

- Makes e-waste disposal organised and easy to track.
- Reward points motivate users to dispose of e-waste responsibly.
- Modular object oriented design: new item categories or user types can be added with little change to existing code.
- Data is saved in files, so records are kept between runs.

10.2 Limitations

- Console-based; there is no graphical interface.
- Data is stored in plain text files rather than a database, and passwords are not encrypted.
- Reward rates are fixed in the program and the points have no real redemption partner yet.

11 Future Scope

- A graphical or mobile application with maps and real-time tracking of the collection vehicle.
- A database back end and secure login.
- Redemption of points with partner shops or recyclers.
- Automatic notifications and reminders for pickups.
- Integration with authorised recyclers to certify that the items were processed correctly.

12 Conclusion

EcoLoop shows how object oriented programming can be used to model a real-world social problem. By representing users, administrators, items, pickups and rewards as cooperating classes, the project applies inheritance, polymorphism, operator and function overloading, static members, templates, exception handling and file handling in one application. The system makes e-waste collection easier to organise and encourages responsible disposal, and its modular design leaves room for future extension.

References

[1] Baldé, C.P., Kuehr, R., Yamamoto, T., et al. *The Global E-waste Monitor 2024*. International Telecommunication Union (ITU) and United Nations Institute for Training and Research (UNITAR), Geneva/Bonn, 2024.

[2] B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013.

[3] E. Balagurusamy, *Object Oriented Programming with C++*, McGraw Hill Education.

[4] cppreference. com – C++ reference documentation, https: //en. cppreference. con.